Observability Fundamentals

monitoring says that, observability finds why

Watch the 4-min video

In one line: monitoring checks the failure modes you predicted and tells you *that* something is wrong; observability lets you ask *new* questions of the logs, metrics and traces you already emit, so you can find out *why*, without shipping new code.

1 Every dashboard is green

It's 3 a.m. Checkout is slow, but only for some users. CPU, memory and error rate all look fine. Your dashboards answer the questions someone predicted when they built them. This problem is one nobody predicted. The test of an observable system: can you explain it from the outside, using data it already produces, without adding a log line and redeploying?

2 Monitoring vs observability

	Monitoring	Observability
Question	Is a known condition true? (up? error rate > 2%? disk full?)	Why is this happening? (questions you didn't plan)
Built from	predefined checks, thresholds, dashboards	rich, linked telemetry you can slice by any field
Good for	known-unknowns, paging on symptoms	unknown-unknowns, debugging novel failures
Output	an alert: *something is wrong*	an explanation: *what, where and why*

You need both. Monitoring is a subset of what good telemetry gives you: alerts (lesson 67) tell you to look; observability lets you find the answer.

3 The three pillars, and how they link

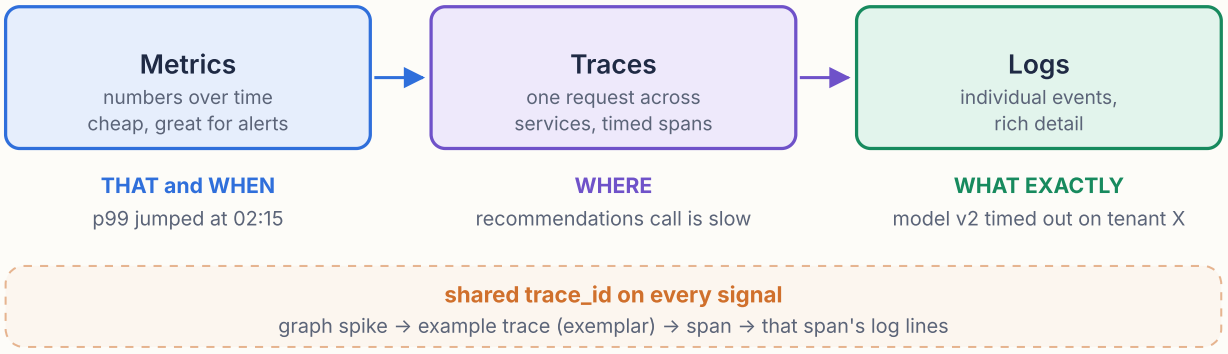


Figure 1. Each signal answers a different question. A shared trace ID turns three separate tools into one investigation.

Signal	What it is	Strength	Weakness	Common tools
Metrics	numeric aggregates over time (rate, p99)	cheap, fast queries, alerting	no per-request detail	Prometheus, Mimir (lesson 62)
Traces	one request's path as a tree of timed spans	shows where time went across services	usually sampled	Jaeger, Tempo (lesson 63)
Logs	timestamped records of single events	full detail and context	volume and cost	Loki, Elasticsearch (lesson 66)

Many teams add **wide events** (one rich record per request) and **continuous profiling** (where CPU and memory go, down to the function) as further signals. The value comes from **linking**: Prometheus *exemplars* attach a trace ID to a histogram sample, so you can click from a latency spike to a real slow trace, then from a span to the log lines that share its trace ID (lesson 65).

4 Cardinality: the limit on metrics

Cardinality is the number of distinct values a field can take. `status` has a handful; `user_id` has millions. A metrics database stores **one time series per unique combination of label values**, so label cardinalities multiply.

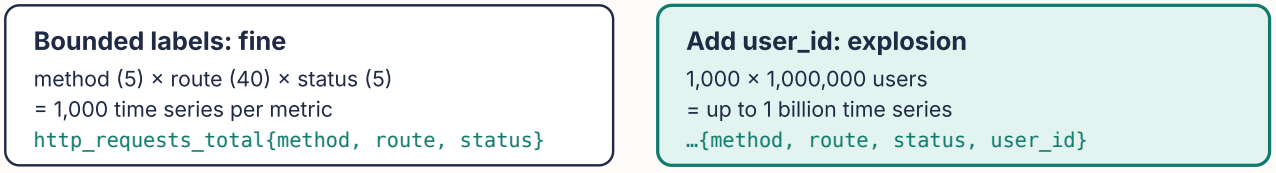


Figure 2. Labels multiply. One unbounded label turns a cheap metric into a memory and cost problem.

The rule: keep metric labels **bounded** (method, route template, status class, region). Put high-cardinality fields such as user ID, request ID and tenant in **logs and traces**, which store each event separately and are built to be searched by them.

5 Building observability in

- **Instrument once** with a vendor-neutral standard, OpenTelemetry (lesson 64): auto-instrumentation gives you HTTP, database and queue spans for free.
- **Emit structured, wide events** with context: tenant, region, service version, feature flag, build ID (lesson 60).
- **Propagate one trace ID** everywhere, including logs and async messages.
- **Keep it queryable** by dimensions you didn't plan: that is exactly what an unknown problem needs.

WORKED EXAMPLE: AN UNEXPLAINED LATENCY SPIKE

p99 checkout latency jumps from **300 ms to 2 s**, and no alert explains it. (1) Open the p99 graph and click an exemplar into a slow trace. (2) The waterfall shows almost all the time in one call: the **recommendations** service. (3) Compare slow traces with fast ones: every slow span carries the attribute `flag=new_ranking`, model version 2. (4) Turn the flag off; p99 returns to 300 ms. No new logging code was needed, because the events already carried the flag:

```
{"event": "checkout", "tenant": "acme", "version": "2.4.0", "flag": "new_ranking", "trace_id": "4bf9…"}
```

COMMON MISTAKES

- Equating observability with "more dashboards": dashboards only answer predicted questions.
- Putting user ID, request ID or full URLs in metric labels, then wondering why Prometheus runs out of memory.
- Three disconnected tools with no shared trace ID, so every investigation starts from scratch in each one.
- Thin events without context (no version, tenant or flag), so you can't slice by the thing that actually changed.

6 Key terms

Monitoring: checking predefined conditions and alerting on them

Telemetry: the logs, metrics, traces and profiles a system emits

Exemplar: a trace ID attached to a metric sample

Observability: ability to explain internal state from external outputs

Cardinality: number of distinct values of a field or label set

Wide event: one structured record per unit of work, with many fields

7 Interview questions

Q1. How is observability different from monitoring? BASIC

Monitoring checks known conditions and tells you something is wrong; observability lets you ask new questions about unknown problems from the telemetry you already emit, so you can find out why.

Q2. What are the pillars of observability? BASIC

Logs, metrics and traces (often plus events and profiles), linked by a shared trace ID, with high-cardinality detail kept in logs and traces rather than metric labels.

Q3. Users report slowness but every dashboard is green. What do you do? SCENARIO

Look at the user-facing symptom directly (p99 by route, region, tenant), jump to slow example traces, find the span that dominates, then compare its attributes with fast requests to see what they have in common.

Q4. A teammate wants a `customer_id` label on the request-latency metric. Your answer? SCENARIO

No: it creates one series per customer and can explode memory and cost. Keep customer ID on spans and logs, and use metrics labels like route and status class.

Q5. If you have great logs, why do you need metrics and traces? TRICKY

Logs are expensive to aggregate over time and don't show causal paths across services; metrics give cheap trends and alerts, traces show where time went in one request.

Q6. Is a system with 500 dashboards observable? TRICKY

Not necessarily: dashboards encode predicted questions. Observability depends on rich, high-cardinality, linked telemetry you can query in new ways.

Next: SD-N-002 · Structured Logging backendroadmap.com